

DirectGCode

Sviluppo di un sistema AI CAM per la programmazione CNC assistita in ambito industriale

**Tesi tecnica indipendente sull'integrazione dell'AI nei processi CAM
industriali**

Lorenzo Marsico

Tecnico industriale e sviluppatore di soluzioni software per automazione, AI applicata
e processi produttivi CNC

Versione pubblicabile - Maggio 2026

Elaborato tecnico-industriale indipendente.

Indice sintetico

1.	Nota di trasparenza	3
2.	Sintesi esecutiva.....	4
3.	Introduzione.....	5
4.	Contesto industriale e requisiti.....	8
5.	Architettura DirectGCode	11
6.	Flusso end-to-end	14
7.	Motore AI CAM	17
8.	Retrieval, RAG e dataset	21
9.	CAM Knowledge e validazione.....	23
10.	Learning e know-how.....	25
11.	G-code e integrazione officina	28
12.	Sicurezza e governance AI.....	30
13.	Test e risultati	32
14.	Industrializzazione e sviluppi futuri	34
15.	Conclusioni.....	36
16.	Appendici	37

Nota di trasparenza e perimetro del documento

Questo documento è una **tesi tecnica indipendente** sull'integrazione dell'intelligenza artificiale nei processi CAM industriali. È un elaborato professionale costruito sulla base di un progetto reale, sviluppato in ambito tecnico-industriale e documentato in forma pubblicabile.

L'obiettivo è descrivere in modo serio, comprensibile e tecnicamente credibile il progetto **DirectGCode**, senza trasformarlo in un manuale del codice sorgente e senza rivelare logiche proprietarie, strategie interne riservate o dettagli implementativi sensibili. La scelta editoriale è volutamente pragmatica: spiegare il problema industriale, l'architettura, il ruolo dell'AI, i meccanismi di controllo e il valore pratico del sistema.

Ho maturato esperienza diretta in officina, automazione, processi CNC, assistenza tecnica e sviluppo software applicato. Questo elaborato nasce da quel contesto: non da una simulazione astratta, ma dal tentativo di costruire uno strumento utilizzabile da chi lavora con macchine, utensili, pezzi reali, tempi ciclo, rischi e responsabilità operative.

Sintesi esecutiva

DirectGCode è un ambiente software per la programmazione CNC assistita, progettato per unire interfaccia operativa, importazione CAD/CAM, motore AI CAM, generazione toolpath, postprocessing G-code, simulazione, apprendimento da casi e supporto decisionale controllato.

Il punto centrale del progetto non è l'idea di sostituire il programmatore CAM con una AI autonoma. Il punto è diverso: trasformare una parte del know-how tecnico in un sistema capace di proporre strategie, utensili, parametri e sequenze di lavorazione, lasciando però all'operatore il controllo finale. In ambito industriale questa distinzione è fondamentale. Una AI che suggerisce può portare valore, una AI che modifica programmi macchina senza controllo può diventare un rischio.

L'architettura è quindi ibrida: combina regole CAM deterministiche, dataset tecnico strutturato, retrieval di casi, knowledge base locale, scoring spiegabile, ranking utensili e feedback operatore. La generazione del codice macchina resta locale e validata. Le funzioni AI non vengono trattate come verità assolute, ma come supporto decisionale. Il sistema cresce nel tempo attraverso casi runtime e feedback, ma mantiene separati dataset base, memoria cliente e conoscenza candidata.

Alla data dell'export tecnico utilizzato per questo documento, il workspace del progetto contiene 2000 condizioni tecniche validate nel dataset AI CAM base, 13184 record candidati nel CAM Knowledge Pack, 39 casi runtime generati, 2 feedback operatore salvati e 67 test mirati superati sui componenti AI CAM principali. Questi numeri non sono presentati come validazione industriale definitiva, ma come fotografia dello stato di avanzamento e della direzione tecnica del progetto.

Il risultato è un modello applicativo di AI industriale orientato alla programmazione CAM locale, controllabile, reversibile, tracciabile e costruito per dialogare con la realtà dell'officina.

1 Introduzione: dal CAM tradizionale all'AI industriale controllata

La programmazione CNC è una delle attività dove il divario tra teoria e pratica è più evidente. Sulla carta il processo sembra lineare: si riceve un disegno, si importa una geometria, si scelgono utensili e parametri, si genera un percorso e si manda il programma in macchina. Nella realtà ogni passaggio contiene esperienza, compromessi, limiti di macchina, disponibilità utensili, rigidità del piazzamento, caratteristiche del materiale, abitudini dell'officina e rischio di errore.

Il CAM tradizionale ha portato un enorme salto di qualità nella produzione meccanica. Ha reso possibile gestire geometrie complesse, ridurre il lavoro manuale sul codice ISO/G-code e standardizzare molte attività. Tuttavia il CAM non elimina il bisogno di competenza. Anzi, spesso lo sposta su un livello superiore come scegliere la strategia corretta, capire quando un parametro è aggressivo, valutare se una profondità è sostenibile, decidere se una feature deve essere lavorata prima o dopo, riconoscere un problema di accessibilità o una condizione di rischio.

In molte officine il vero patrimonio non è solo il parco macchine, ma il know-how delle persone. Il problema è che questo patrimonio è spesso non strutturato. Rimane nella testa degli operatori esperti, nei file CAM salvati negli anni, in programmi G-code modificati a mano, in note informali, in prove riuscite e in errori che nessuno vuole ripetere. Quando una persona cambia ruolo, va in pensione o semplicemente non è disponibile, una parte della conoscenza produttiva si indebolisce.

L'intelligenza artificiale può aiutare, ma solo se viene applicata con criterio industriale. Un modello generativo puro, lasciato libero di inventare una strategia CAM, non è sufficiente. In officina non basta che una risposta sia plausibile: deve essere coerente con la macchina, con l'utensile, con il materiale, con il grezzo, con il controllo CNC e con il rischio reale di lavorazione. Per questo DirectGCode non nasce come chatbot, ma come architettura AI CAM ibrida.

1.1 Obiettivo del progetto

L'obiettivo di DirectGCode è supportare la programmazione CNC attraverso un ambiente operativo capace di:

- importare o descrivere una geometria di lavoro;
- analizzare il contesto produttivo;
- riconoscere feature lavorabili;
- proporre strategie CAM coerenti;
- suggerire utensili e parametri di taglio;
- recuperare casi simili già appresi;
- generare toolpath e G-code in modo locale;
- lasciare all'operatore la possibilità di confermare, correggere o rifiutare ogni scelta;
- memorizzare l'esperienza utile per migliorare le proposte future.

Il sistema non vuole annullare la figura del programmatore. Vuole aumentarne la capacità operativa. L'operatore resta responsabile della decisione finale, ma può contare su una base tecnica strutturata

che riduce il tempo di ricerca, aiuta a non dimenticare passaggi critici e rende più riutilizzabile la conoscenza generata nel tempo.

1.2 Perché un approccio locale

Un punto progettuale importante è la scelta di mantenere il cuore operativo locale. In ambiente industriale non sempre è accettabile dipendere da una connessione cloud per generare o validare un programma CNC. Le ragioni sono diverse: continuità produttiva, riservatezza dei dati cliente, protezione del know-how, sicurezza informatica, latenza, responsabilità sul risultato e disponibilità dell'impianto.

DirectGCode separa quindi ciò che può essere eventualmente esposto come servizio da ciò che deve rimanere sotto controllo locale. La GUI desktop e la generazione operativa restano locali. Le API e la containerizzazione sono trattate come componenti opzionali e progressivi, non come requisito per far funzionare la produzione.

1.3 Il ruolo dell'AI

Nel progetto l'AI non è un'autorità assoluta. È un motore di supporto. Propone, confronta, recupera casi, assegna confidenza, evidenzia warning e suggerisce correzioni prudenziali. La decisione critica resta human-in-the-loop: l'operatore vede la proposta, modifica se necessario e solo dopo porta il risultato verso la generazione o l'esportazione.

Questa impostazione è meno spettacolare rispetto a una narrazione in cui la AI "fa tutto da sola", ma è molto più adatta al contesto industriale. Una lavorazione CNC non è una risposta testuale, è un'azione fisica su materiale, macchina e utensile. Il margine di errore non è virtuale.

1.4 Dalla teoria alla responsabilità operativa

La differenza tra un progetto dimostrativo e un progetto industriale sta nella responsabilità dell'output. Un algoritmo che sbaglia una classificazione in un ambiente sperimentale produce un errore statistico; un sistema che suggerisce una lavorazione errata può produrre una collisione, un utensile rotto, un pezzo da scartare o un fermo macchina. Questo cambia completamente il modo in cui l'AI deve essere progettata.

Per questo DirectGCode non parte dall'idea di "automatizzare tutto", ma dall'idea di costruire un percorso decisionale controllato. L'AI entra dove può aumentare la qualità della decisione: confronto con casi precedenti, recupero di pattern, suggerimento di parametri, rilevamento di incoerenze e presentazione di warning. Non entra invece come autorità cieca sull'output finale.

Questo approccio è meno appariscente, ma più solido. In officina il problema non è stupire l'operatore con una proposta brillante; il problema è costruire fiducia progressiva. La fiducia nasce quando il sistema è prevedibile, spiega le sue scelte, permette la correzione e conserva memoria degli esiti.

1.5 Valore pratico atteso

Il valore di un sistema come DirectGCode può essere letto su più livelli. Il primo è operativo: ridurre il tempo necessario a preparare un lavoro, soprattutto quando il programmatore deve partire da zero. Il secondo è organizzativo: rendere più riutilizzabile l'esperienza già accumulata. Il terzo è formativo: aiutare operatori meno esperti a ragionare con il supporto di casi, warning e parametri coerenti. Il quarto è strategico: creare una base di conoscenza tecnica che rimane nell'azienda.

Questi benefici non devono essere misurati solo in minuti risparmiati. In produzione conta anche ridurre l'incertezza. Un sistema che ricorda casi simili, evidenzia un rischio o suggerisce un controllo può evitare errori difficili da quantificare preventivamente. La vera efficienza non è soltanto velocità, ma qualità della decisione.

2 Contesto industriale e requisiti reali

Ogni progetto di AI applicata all'industria dovrebbe partire da una domanda semplice: quale problema reale si vuole risolvere? Nel caso di DirectGCode il problema non è generare automaticamente righe di codice, ma costruire un processo di programmazione più assistito, più tracciabile e meno dipendente dalla memoria individuale.

2.1 La programmazione CNC come processo decisionale

Programmare una macchina CNC significa prendere molte decisioni concatenate. Alcune sono evidenti: materiale, utensile, velocità mandrino, avanzamento, profondità di passata. Altre sono meno visibili ma spesso più importanti: ordine delle operazioni, strategia di ingresso, gestione del sovrametallo, accessibilità, rischio di vibrazione, controllo della qualità e possibilità di recupero in caso di errore.

In un flusso reale, il programmatore non sceglie semplicemente “una tasca” o “una contornatura”. Valuta il pezzo, il grezzo, la macchina disponibile, la libreria utensili, il tempo richiesto, la robustezza del piazzamento e il grado di finitura necessario. Due geometrie simili possono richiedere scelte diverse se cambiano materiale, utensile disponibile o macchina.

Per questo l'AI CAM deve considerare il contesto. Un suggerimento valido in astratto può essere sbagliato nella realtà produttiva. DirectGCode tratta il contesto come parte integrante della decisione: macchina, assi disponibili, interfaccia mandrino, materiale, grezzo, utensile e origine non sono dati secondari, ma vincoli.

2.2 Requisiti principali

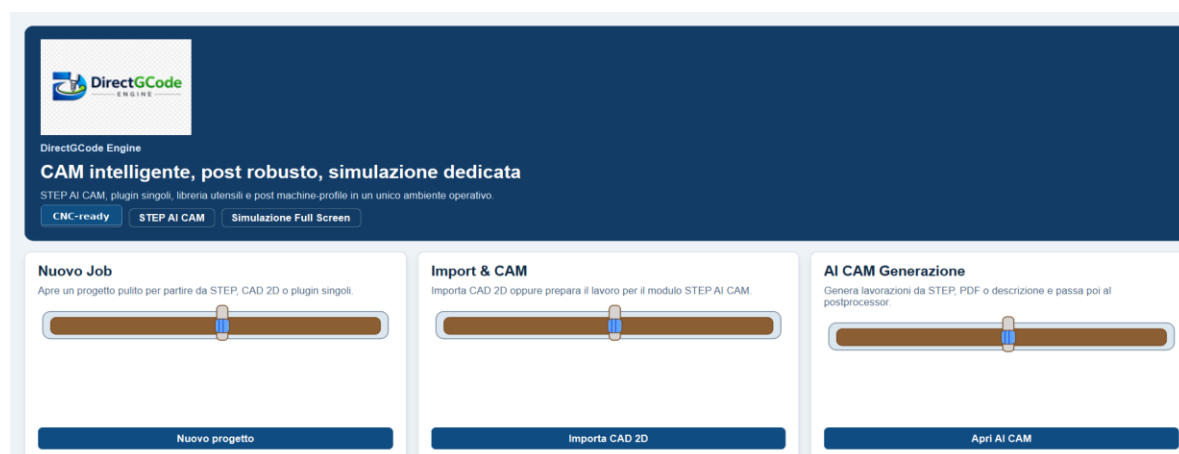
I requisiti industriali che guidano il progetto possono essere riassunti in sette aree.

Area	Requisito	Motivazione
Continuità operativa	Funzionamento locale del desktop operativo	La produzione non deve fermarsi per indisponibilità cloud o rete esterna
Controllo umano	Conferma operatore per output critici	Il G-code è un output fisico e non solo informativo
Tracciabilità	Memorizzazione di suggerimenti, modifiche e feedback	Serve capire cosa è stato proposto e cosa è stato corretto
Reversibilità	Separazione tra proposte AI, dati base e feedback	Un apprendimento errato non deve contaminare il dataset principale
Generalità macchina	Riferimenti a centro di lavoro CNC e profili macchina	Il progetto non deve essere legato a un marchio specifico
Sicurezza dati	Separazione tra dati cliente, learning locale e conoscenza globale	Il know-how dell'officina deve rimanere protetto
Industrializzazione	API e container opzionali	Il sistema deve poter crescere senza rompere il desktop esistente

2.3 Macchina agnostica, non marchio dipendente

Un aspetto importante della versione pubblicabile è la scelta di descrivere il sistema come applicabile a un **centro di lavoro CNC** generico. Nel progetto reale una macchina disponibile in officina può essere stata utilizzata come banco di prova, ma questo non deve trasformarsi in un vincolo concettuale. Il valore del sistema sta nella capacità di adattarsi a profili macchina differenti, controlli diversi e modalità di trasferimento configurabili.

Per questo documento ogni riferimento operativo viene mantenuto sul piano generale: centro di lavoro CNC, controllo CNC, postprocessor locale, trasferimento programma e validazione ISO/G-code. In questo modo DirectGCode viene presentato come piattaforma CAM intelligente generalizzabile, non come accessorio di una macchina specifica.



Dashboard operativa del sistema, con accesso ai moduli principali di importazione, AI CAM e generazione.

2.4 Vincoli non scritti di officina

Molti vincoli importanti non sono presenti nel disegno tecnico. Il disegno dice cosa deve essere prodotto, ma non sempre dice come conviene produrlo nella specifica officina. La scelta reale dipende da utensili disponibili, stato macchina, rigidità dello staffaggio, esperienza sugli stessi materiali, urgenza del lavoro, priorità produttive e abitudini consolidate.

Questi vincoli non scritti sono difficili da modellare. Eppure sono proprio quelli che distinguono una proposta teorica da una proposta utilizzabile. DirectGCode affronta il problema creando una memoria di casi e feedback: non pretende di conoscere tutto all'inizio, ma costruisce un percorso per trasformare esperienza pratica in dati tecnici riutilizzabili.

2.5 Attori del processo

Il sistema coinvolge più ruoli, anche quando in una piccola realtà una sola persona ne svolge diversi:

Ruolo	Esigenza principale	Come DirectGCode può supportare
Programmatore CAM	Preparare strategie coerenti e velocemente	Recupero casi, suggerimento utensili, warning, tabella modificabile

Ruolo	Esigenza principale	Come DirectGCode può supportare
Operatore macchina	Ricevere programma e istruzioni chiare	Setup sheet, anteprima toolpath, parametri leggibili
Responsabile produzione	Ridurre tempi morti e dipendenza da singoli esperti	Memoria tecnica, standardizzazione, tracciabilità
Responsabile qualità	Capire origine delle scelte e controlli previsti	Note, warning, dati progetto e feedback
Sviluppatore/maintainer	Evolgere il software senza rompere il flusso	Feature flag, test, separazione dei servizi

Questa lettura è importante perché un sistema AI industriale non vive solo nel computer. Vive nell'interazione con persone, processi, macchine e responsabilità.

2.6 Benefici misurabili e benefici indiretti

Alcuni benefici possono essere misurati con indicatori relativamente semplici: tempo di programmazione, numero di correzioni manuali, tempo ciclo stimato, numero di warning, percentuale di proposte confermate, frequenza di riutilizzo dei casi. Altri benefici sono più indiretti: maggiore uniformità tra programmatori, riduzione della perdita di know-how, migliore preparazione della documentazione officina, maggiore velocità di inserimento di nuovi operatori.

DirectGCode deve quindi essere valutato non solo come generatore di codice, ma come sistema di supporto al processo. Se riduce il tempo di programmazione ma aumenta il rischio, non ha valore. Se invece migliora la qualità delle decisioni e mantiene il controllo umano, può diventare uno strumento realmente utile.

3 Architettura generale di DirectGCode

DirectGCode è organizzato come un'applicazione desktop con servizi AI/CAM, motore core di generazione, knowledge base locale e componenti headless/API opzionali. Questa scelta architettuale nasce da una considerazione pratica: l'interfaccia deve essere usabile in officina, ma il motore deve essere abbastanza modulare da poter evolvere verso scenari più industrializzati.

L'applicazione non è concepita come un singolo blocco monolitico. La logica viene suddivisa in livelli: interfaccia utente, servizi AI/CAM, core CAM, knowledge storage, validazione, generazione G-code e API opzionale. Questa separazione permette di far crescere il progetto senza riscrivere ogni volta tutto il sistema.

3.1 Livello interfaccia

Il livello UI è basato su un'applicazione desktop. Il suo compito è rendere utilizzabile il motore tecnico: aprire progetti, importare geometrie, mostrare pannelli AI CAM, visualizzare toolpath, gestire salvataggi, esporre tabelle modificabili e permettere all'operatore di intervenire.

La GUI non è un elemento secondario. In un sistema human-in-the-loop l'interfaccia è parte del controllo. Se l'operatore non capisce cosa sta proponendo la AI, non può confermare in modo responsabile. Per questo il progetto prevede pannelli dedicati alla generazione AI CAM, al learning, alla visualizzazione STEP e al canvas toolpath.

3.2 Livello servizi AI/CAM

Il livello servizi contiene le funzioni di pianificazione, apprendimento, retrieval, ranking utensili, calcolo parametri, decision support e gestione knowledge. In questo livello DirectGCode costruisce la propria intelligenza operativa: non come modello unico e opaco, ma come insieme di servizi specializzati che collaborano.

La scelta è importante. Un solo modello che produce direttamente il G-code sarebbe più semplice da raccontare, ma molto meno controllabile. DirectGCode preferisce una pipeline più articolata: ogni servizio produce una parte della decisione e rende possibile inserire controlli, warning, fallback e override.

3.3 Livello core CAM

Il core CAM si occupa della generazione dei toolpath, del postprocessing G-code, della validazione progetto e della stima tempi. È la parte che traduce una decisione tecnica in un output operativo.

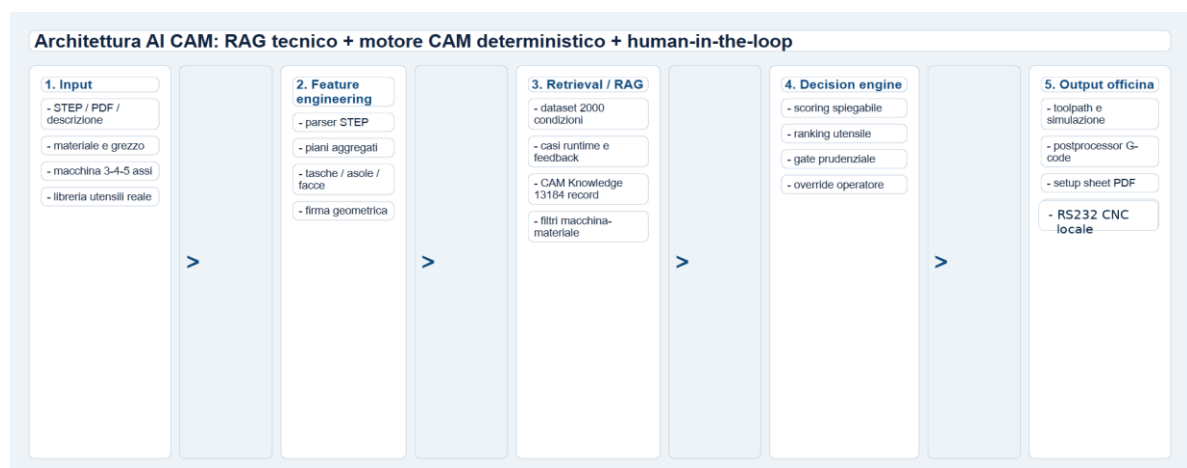
In questa area il principio è mantenere una base deterministica. L'AI può suggerire una strategia, ma il toolpath e il G-code devono passare da componenti controllabili, verificabili e validabili. Questo evita che la parte generativa diventi una scatola nera non ispezionabile.

3.4 Livello headless/API opzionale

Il progetto prevede anche un confine headless e un'API opzionale. Questa componente permette di esporre alcune funzioni come validazione, stima tempo e generazione da geometrie semplici. Tuttavia non tutto viene portato automaticamente dentro API o Docker.

La scelta è prudentiale: le funzioni critiche legate alla produzione locale, alla GUI e al trasferimento macchina rimangono nel desktop. Le API sono pensate per industrializzare gradualmente il motore, non per sostituire subito il flusso operativo.

Livello	Funzione	Criterio progettuale
UI desktop	Operatività, pannelli, visualizzazione, modifica manuale	L'operatore deve vedere e controllare
Servizi AI/CAM	Suggerimenti, retrieval, scoring, learning	La AI deve assistere, non comandare
Core CAM	Toolpath, postprocessor, validazione, stima tempi	La generazione deve rimanere deterministica
Knowledge base	Dataset, casi runtime, feedback, knowledge pack	La conoscenza deve essere separata e tracciabile
API opzionale	Funzioni headless conservative	Industrializzazione progressiva e reversibile



Schema architetturale del flusso AI CAM: input, feature engineering, retrieval, decision engine e output officina.

3.5 Principio di separazione

La scelta architetturale più importante è la separazione tra proposta, validazione e generazione. In un sistema CAM tradizionale questi passaggi possono essere percepiti come parte dello stesso flusso. In un sistema AI CAM devono essere distinti con più attenzione.

La proposta AI è il risultato di retrieval, scoring, dataset e regole. La validazione controlla coerenza, limiti e rischi. La generazione produce toolpath e G-code attraverso componenti deterministici. Il salvataggio registra l'esito e può alimentare il learning. Separare questi passaggi permette di intervenire su una parte senza compromettere le altre.

3.6 Artefatti gestiti dal sistema

Il progetto lavora con diversi artefatti tecnici:

Artefatto	Significato	Rischio se gestito male
Geometria STEP/CAD	Forma del pezzo o feature da lavorare	Interpretazione errata della lavorazione
Parametri job	Materiale, grezzo, macchina, utensili, origine	Proposte non compatibili con la realtà
Piano AI CAM	Operazioni e strategie suggerite	Fiducia eccessiva in una proposta non validata
Toolpath	Percorso utensile	Collisioni, traiettorie non previste, inefficienza
G-code	Output macchina	Movimento errato, fermo macchina o scarto
Setup sheet	Informazioni per officina	Errori di attrezzaggio o comunicazione incompleta
Feedback	Correzioni e conferme dell'operatore	Apprendimento distorto se non separato e validato

Questa varietà di artefatti spiega perché il progetto non può essere ridotto a “generazione testo”. Il codice macchina non è un testo qualsiasi: è un file operativo che deve corrispondere a una realtà fisica.

3.7 Configurazione come elemento industriale

In un software di officina la configurazione è parte del prodotto. Profili macchina, librerie utensili, limiti mandrino, modalità di esportazione, feature flag e percorsi dati devono essere gestiti in modo ordinato. Una buona architettura non richiede di modificare il codice sorgente ogni volta che cambia una macchina o un parametro operativo.

DirectGCode va quindi interpretato anche come piattaforma configurabile. La generalizzazione a diversi centri di lavoro CNC dipende da questo: non hardcodare una macchina, ma descrivere profili e vincoli.

4 Flusso operativo end-to-end

Il flusso operativo di DirectGCode può essere letto come una catena di trasformazione controllata. L'input iniziale può arrivare da una geometria STEP, da un PDF tecnico, da una descrizione testuale o da una geometria manuale. Il sistema raccoglie il contesto produttivo, costruisce una proposta AI CAM, consente la revisione dell'operatore, genera il toolpath e produce l'output officina.

4.1 Dall'input al piano di lavorazione

Il processo inizia con la definizione del lavoro. L'operatore imposta materiale, grezzo, profilo macchina, controllo CNC, utensili disponibili, origine e parametri base. Questa fase non è una formalità: serve a impedire che la AI ragioni su condizioni non realistiche.

Quando viene importato un modello o una descrizione, il sistema non passa immediatamente alla generazione del G-code. Prima costruisce una rappresentazione lavorabile del problema. Nel caso di un modello STEP, ad esempio, vengono analizzate le superfici, i piani, le feature e le possibili regioni di lavorazione.

Il risultato è un piano AI CAM composto da operazioni, strategie, utensili suggeriti, profondità, velocità, avanzamenti, warning e riferimenti a casi appresi. Questo piano non è ancora un programma definitivo: è una proposta strutturata.

4.2 Revisione operatore

La proposta viene esposta in tabella. L'operatore può modificare profondità, avanzamento, ingresso, strategia, utensile e altri valori. Questo è un passaggio centrale: il sistema non nasconde la proposta e non forza automaticamente l'adozione. L'operatore può accettare, correggere o forzare una scelta.

La differenza rispetto a un CAM tradizionale non è che il programmatore sparisce, ma che il programmatore riceve un supporto più strutturato. Vede una proposta coerente con i casi disponibili, con la knowledge base e con il dataset tecnico. Può decidere più velocemente e con più informazioni.

4.3 Generazione e output

Dopo la revisione, la generazione passa attraverso toolpath generator e postprocessor locale. Il G-code viene validato, stimato come tempo ciclo, visualizzato e infine esportato. Il sistema può generare anche setup sheet e pacchetti informativi per l'officina.

Il trasferimento verso macchina CNC, quando previsto, resta una fase distinta e validata. L'obiettivo è evitare che una proposta AI arrivi direttamente alla macchina senza controlli intermedi.

AI CAM Generazione
 Genera lavorazioni AI CAM partendo da STEP, descrizioni testuali o parametri inseriti a mano. L'apprendimento resta separato, ma viene richiamato in background per scegliere ricette, utensili e casi simili già appresi.

Apri STEP AI CAM

Genera da PDF

Genera da descrizione

Genera GCODE con AI CAM

Applica modifiche tabella

Esporta GCODE

Esporta Setup Sheet PDF

Esporta GCODE + Setup Sheet

Prompt lavorazione AI CAM

Esempio tesi: genera una lavorazione AI CAM per profilo encoder, materiale Acciaio C40, macchina 3 assi ISO40, con controllo feed/S e verifica CAM Knowledge.

Esito generazione

Progetto demo: Pista encoder smussatrici
 Materiale: Acciaio C40
 Dataset tecnico: 2000 condizioni validate
 Runtime learning: 39 casi + 2 feedback operatore

Casi appresi richiamati

Caso library totale: 157
 Pattern atomici disponibili: 29
 Knowledge pack installato: 13184 record candidati
 Retrieval filtrato per materiale, macchina, interfaccia mandrino e strategia.

Operazioni AI CAM del job

Op	Strategia	Origine AI	Caso appreso	Utensile	Profondità	Velocità/rotazione	Avanzamento lavoro	Step Z	Ingresso	Warning
1	contornatura spallamento	Guidato da learning	dataset/base	Fresa integrale D16.0 Z4	-16.12	2224	456.0	16.12	Auto	controllo setup e simulazione richiesti

Proposte CAM Knowledge

Record	Stato	Score	Operazione	Utensile	Rischi	Controlli
DGK00057	candidate	0.82	Maschiatura a rullare	maschio formatore	coppia eccessiva, materiale non idoneo	verificare materiale; controllare preforo
DGK00058	candidate	0.78	Maschiatura a rullare	maschio formatore	coppia eccessiva, materiale non idoneo	verificare materiale; controllare preforo
DGK00059	candidate	0.75	Maschiatura a rullare	maschio formatore	coppia eccessiva, materiale non idoneo	verificare materiale; controllare preforo

Applica proposta al progetto

Conferma operatore

Valida e attiva al salvataggio

Operazioni | Controllo AI | Pre-post

Operazioni AI presenti nel job: 1
 Macchina target: 3 assi / ISO40
 G-code stimato/generabile tramite postprocessor locale.

Pannello AI CAM per la generazione delle operazioni, con tabella modificabile, casi richiamati e proposte knowledge.

4.4 Catena di controllo

Il flusso end-to-end può essere visto come una catena di gate. Ogni gate riduce il rischio prima del passaggio successivo:

1. **Gate contesto:** verifica che materiale, grezzo, utensile e profilo macchina siano presenti.
2. **Gate geometrico:** verifica che la feature o il modello siano interpretabili.
3. **Gate AI CAM:** propone operazioni con confidenza e warning.
4. **Gate operatore:** consente correzione e conferma umana.
5. **Gate CAM deterministico:** genera toolpath e G-code con regole controllate.
6. **Gate validazione:** controlla coerenza del programma.
7. **Gate officina:** esporta setup sheet e programma pronto per l'uso.

Questa catena non elimina il rischio, ma lo rende più visibile. Un processo industriale sicuro non promette assenza di errore: costruisce punti di controllo in cui l'errore può essere intercettato.

4.5 Traccia decisionale

Ogni proposta AI CAM dovrebbe poter essere riletta attraverso una traccia decisionale: da quale input nasce, quali casi sono stati richiamati, quali vincoli sono stati considerati, quali warning sono emersi e quali modifiche ha applicato l'operatore. Questa traccia è utile sia per migliorare il sistema sia per discutere tecnicamente una decisione.

In prospettiva, la traccia decisionale può diventare uno strumento di qualità. Non sostituisce la misura del pezzo o il controllo finale, ma permette di capire meglio come si è arrivati a una determinata scelta produttiva.

5 Motore AI CAM

Il motore AI CAM è il cuore del progetto. La sua funzione è trasformare un input tecnico in una proposta di lavorazione utilizzabile. Per farlo combina feature engineering geometrico, dataset tecnico, retrieval di casi, calcolo parametri, ranking utensili, scoring e decision support.

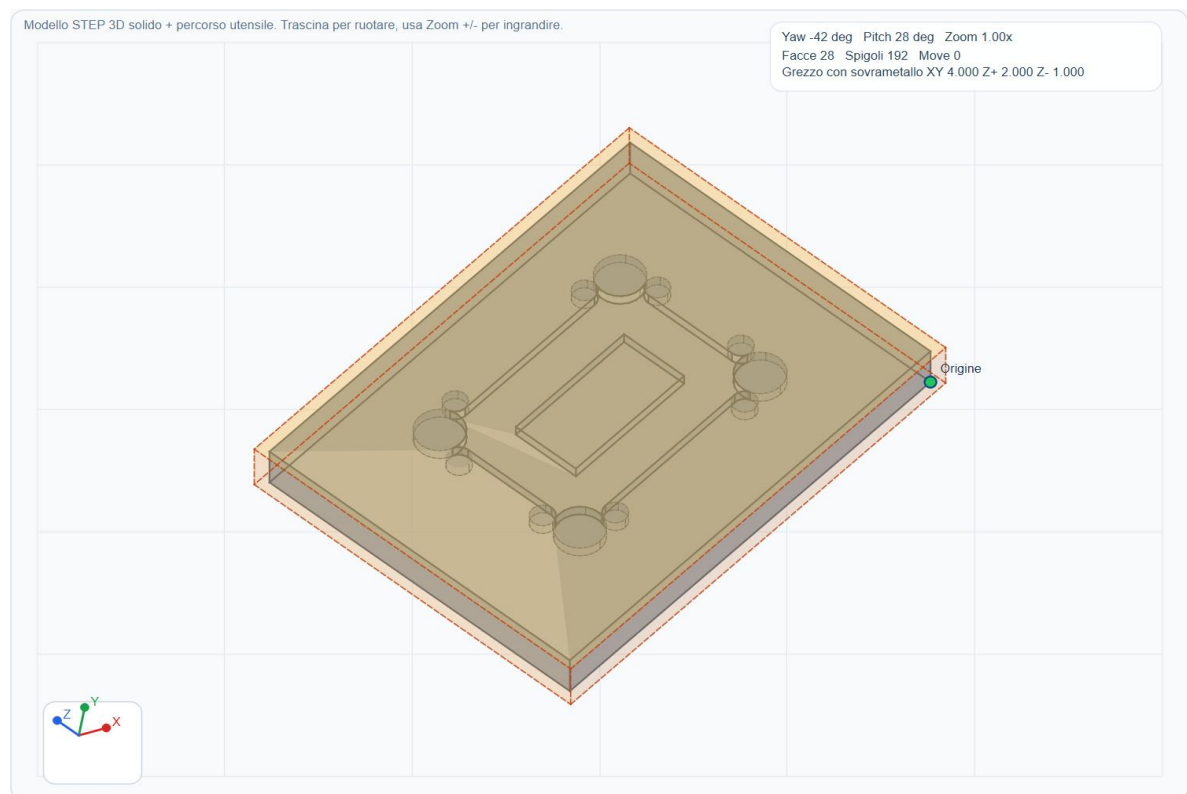
La parola “AI” in questo contesto non indica un singolo modello che decide tutto. Indica un sistema composto da più componenti. Alcune parti sono regole CAM deterministiche, altre sono modelli o euristiche di ranking, altre ancora sono recupero di casi e confronto con knowledge base.

5.1 STEP AI CAM

Quando l’input è un modello STEP, il sistema effettua una pipeline di analisi geometrica. Il modello viene validato, letto e trasformato in elementi tecnici. Vengono rilevati piani, facce, regioni, tasche e superfici utili. Il sistema prova a comprendere quali zone possano diventare feature lavorabili.

Questa fase è essenziale perché un retrieval basato solo su testo sarebbe troppo debole. In CAM la geometria conta. Una tasca, una faccia planare, una regione aperta o un piano laterale non sono semplici parole: hanno dimensioni, profondità, accessibilità, orientamento e relazione con il grezzo.

Il risultato è un piano AI CAM che contiene il modello, il materiale, l’utensile, il grezzo, le velocità, gli avanzamenti, le operazioni, le feature riconosciute, il setup plan e i riferimenti alla decision support.



Viewer STEP 3D con modello solido, grezzo, origine e anteprima del contesto geometrico.

5.2 Feature engineering geometrico

Il feature engineering è una parte spesso sottovalutata. Molti discorsi sull'AI industriale partono dal modello, ma in realtà la qualità del risultato dipende moltissimo da come viene costruito l'input. Se la geometria non viene rappresentata correttamente, anche il miglior modello rischia di produrre suggerimenti deboli.

Nel progetto vengono considerate proprietà come:

- planarità delle superfici;
- area minima lavorabile;
- connessione tra facce;
- normale del piano;
- bounding box;
- posizione rispetto al grezzo;
- ruolo della superficie nel pezzo;
- profondità e accessibilità.

Queste informazioni permettono di costruire una firma geometrica. La firma non serve a identificare il pezzo in senso commerciale, ma a confrontare feature simili. Una tasca profonda e chiusa non deve essere trattata come una semplice faccia da spianare. Una regione aperta sul bordo ha rischi e strategie diverse rispetto a una tasca interna.

5.3 Setup planning

La pianificazione del setup valuta come il pezzo può essere affrontato dalla macchina. Non tutte le feature sono lavorabili nello stesso modo o dallo stesso orientamento. Il sistema deve riconoscere candidati di setup, piani principali e aree che richiedono attenzione.

Anche qui la AI non opera da sola. Il setup planning combina informazioni geometriche e regole. L'obiettivo è proporre una sequenza ragionevole, non sostituire la valutazione dell'attrezzaggio reale, che rimane un fattore fisico dipendente da morsa, staffaggio, accessibilità e rigidità.

5.4 Calcolo parametri di taglio

Il calcolo di velocità mandrino e avanzamento parte dalla relazione base tra velocità di taglio, diametro utensile, numero taglienti e avanzamento per dente:

$$S = \frac{(1000 * Vc_effettiva)}{(pi * D)}$$

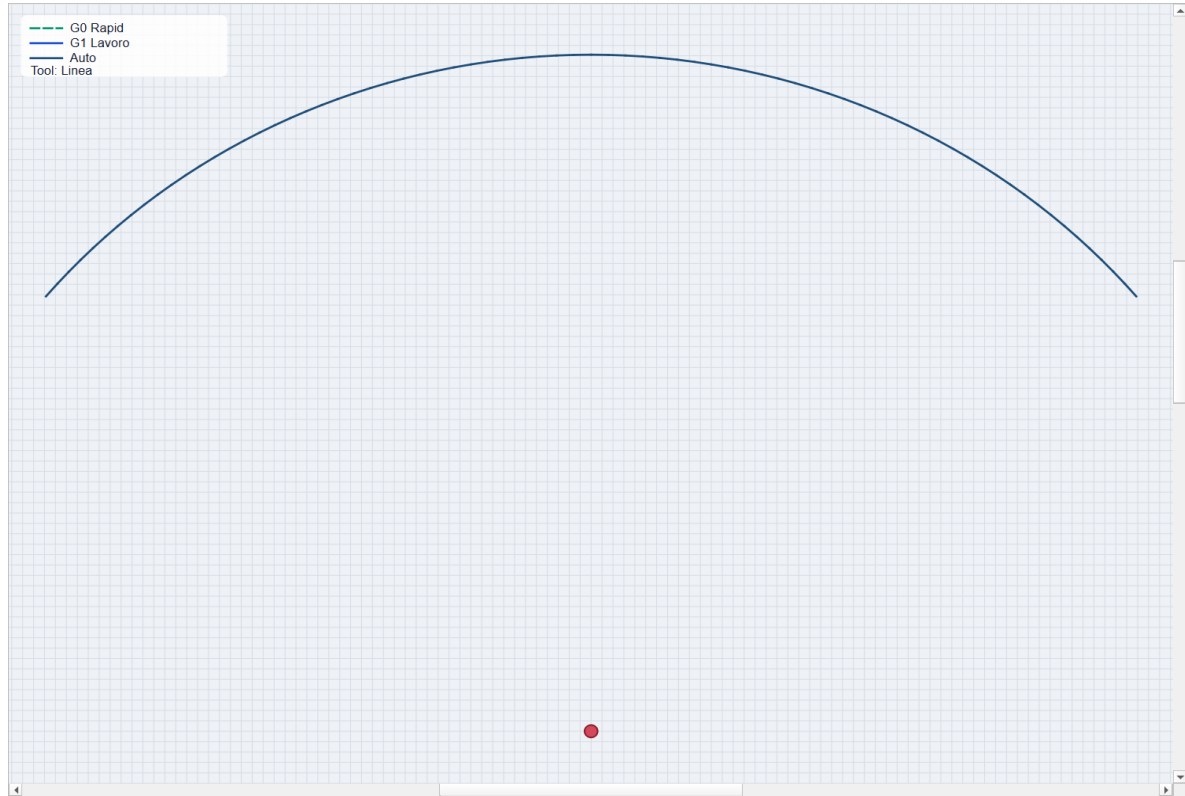
$$F = S * denti_effettivi * fz_effettivo$$

Questa formula non è il punto innovativo in sé: è una base tecnica nota. Il valore sta nel modo in cui il sistema applica correzioni e warning in funzione del contesto. Materiale, utensile, costruzione, lubrificazione, strategia, profondità e limite mandrino possono modificare la proposta.

Il sistema può segnalare condizioni come utensile non valido, profondità incoerente, avanzamento eccessivo, step Z fuori scala o override manuale. Questi warning sono importanti perché rendono la proposta più prudente e leggibile.

5.5 Toolpath e simulazione

La generazione del toolpath resta in un livello controllato. Il motore può ricostruire in memoria percorsi, simulare traiettorie e produrre output compatibile con il postprocessor. La visualizzazione del percorso permette all'operatore di verificare graficamente ciò che il sistema intende fare.



Canvas 2D con anteprima del toolpath e distinzione tra movimenti rapidi e movimenti di lavoro.

5.6 Strategia senza rivelare il segreto

In un documento pubblicabile è importante spiegare il funzionamento del motore senza rivelare dettagli proprietari. La distinzione è questa: si può descrivere la logica industriale del sistema, ma non è necessario esporre codice sorgente, soglie interne, algoritmi specifici, pesi completi o ricette riservate.

La tesi tecnica del motore è chiara: il sistema osserva geometria e contesto, costruisce feature lavorabili, confronta casi, calcola parametri, assegna confidenza, propone una sequenza e mantiene il controllo umano. Questo basta al lettore per comprendere il valore del progetto senza trasformare il documento in una divulgazione del know-how interno.

5.7 Dalla feature alla decisione

Una feature non diventa automaticamente un'operazione. Il sistema deve passare da una lettura geometrica a una scelta CAM. Questo passaggio include domande come:

- la feature è accessibile con la configurazione macchina disponibile?
- quale utensile è coerente con dimensione e profondità?

- la lavorazione richiede sgrossatura, finitura o entrambe?
- l'ingresso utensile è sicuro?
- la profondità proposta è compatibile con utensile e materiale?
- esistono casi simili già appresi?
- il rischio è basso, medio o richiede conferma esplicita?

La risposta non è una singola formula. È il risultato di più elementi combinati. Proprio per questo il sistema deve restituire non solo un valore, ma anche un contesto di decisione.

5.8 Warning come parte del valore

In molti sistemi software il warning viene percepito come fastidio. In DirectGCode è parte del valore. Un warning corretto può evitare un errore costoso. Il sistema non deve solo proporre “cosa fare”, ma anche indicare “cosa controllare”.

Esempi di warning utili sono: utensile non coerente con la profondità, parametri mancanti, avanzamento eccessivo, strategia aggressiva, setup non verificato, entry mode da controllare, o discrepanza tra scelta operatore e ranking suggerito. Anche quando l'operatore decide di procedere, la presenza del warning rende la scelta consapevole.

6 Retrieval, RAG tecnico e dataset

Una delle caratteristiche più interessanti di DirectGCode è l'uso di un approccio RAG-like applicato al CAM. Nel linguaggio comune il RAG viene spesso associato al recupero di documenti testuali. In questo progetto il concetto viene adattato al contesto tecnico: il sistema recupera casi, pattern, condizioni e record CAM pertinenti per guidare la proposta di lavorazione.

6.1 Dataset tecnico base

Il dataset AI CAM base contiene 2000 condizioni tecniche validate. Ogni condizione include informazioni su materiale, macchina, setup, utensile, operazione, strategia, target, criticità, quality check e tag di apprendimento. Il dataset viene normalizzato in funzione di configurazione assi, interfaccia mandrino, famiglie materiali, gruppi ISO, famiglia operazione, feature type, utensile, entry mode e pattern toolpath.

Il dataset non è trattato come una lista piatta. Viene trasformato in una case library, cioè una libreria di casi tecnici utilizzabili in inference. Ogni caso contiene strategia, confidenza, evidence score, fingerprint geometrico, pattern key, fonte e vincoli.

Questa impostazione permette al sistema di cercare casi compatibili con il lavoro corrente. Non basta che due casi abbiano lo stesso materiale: devono essere compatibili anche con macchina, utensile, feature e contesto.

6.2 Retrieval dei casi

Nel flusso operativo il sistema costruisce un contesto RAG che include:

- campioni CAM professionali importati;
- casi runtime generati dal sistema;
- feedback operatore;
- dataset base;
- CAM Knowledge Pack;
- vincoli macchina;
- materiale del job;
- firma geometrica del modello, quando disponibile.

Il retrieval non deve produrre una risposta creativa, ma una selezione di evidenze. I casi recuperati aiutano a proporre strategia, utensile, profondità, ingresso e warning. Se il modello corrente risulta molto simile a un caso appreso, il sistema può adottare una modalità più fedele al riferimento, mantenendo però adattamenti prudenziali.

6.3 Scoring spiegabile

Lo scoring ha un ruolo pratico: deve spiegare perché un caso o una strategia viene proposta. Il sistema considera elementi come compatibilità materiale, assi macchina, interfaccia, classe utensile, rischio, lifecycle e confidenza del caso. Questo rende il suggerimento più leggibile.

In un sistema industriale la spiegabilità non è un lusso. Se il programmatore vede solo una raccomandazione senza motivazione, difficilmente potrà fidarsi. Se invece vede che la proposta deriva da materiale compatibile, feature simile, utensile coerente e caso validato, può valutarla in modo più rapido.

6.4 Machine learning in shadow mode

Il ranking utensili può funzionare in modalità shadow. Questo significa che il sistema può confrontare la scelta corrente con una raccomandazione, ma non forza automaticamente la sostituzione. È una scelta intelligente per una fase di crescita: il modello viene osservato, confrontato e migliorato senza trasformarsi in autorità produttiva.

Quando il provider ML non è disponibile, il sistema può utilizzare un fallback euristico. Questo evita che una componente AI opzionale diventi un punto singolo di fallimento. La robustezza industriale si costruisce anche così: ogni parte deve avere un comportamento prevedibile in caso di indisponibilità.

6.5 Differenza tra RAG testuale e RAG industriale

Nel RAG testuale classico il sistema recupera documenti e li usa per rispondere a una domanda. Nel RAG industriale applicato al CAM il recupero deve essere più vincolato. Non basta trovare una frase simile: serve trovare condizioni operative compatibili.

Una strategia appresa su un materiale diverso, con utensile diverso e macchina diversa può essere fuorviante. Una geometria simile ma con profondità differente può richiedere parametri completamente diversi. Per questo DirectGCode non usa il retrieval come semplice ricerca di testo, ma come confronto di casi tecnici.

6.6 Evidenza, non magia

Il retrieval deve produrre evidenza. Un caso richiamato è utile se spiega perché è pertinente. La pertinenza può derivare da materiale, feature, dimensioni, profondità, utensile, setup, pattern o conferme precedenti. Quando l'evidenza è debole, il sistema deve abbassare la confidenza o chiedere più controllo umano.

Questa impostazione impedisce al sistema di presentare raccomandazioni deboli come se fossero certe. In produzione, l'incertezza deve essere mostrata, non nascosta.

6.7 Dataset base e memoria locale

Il dataset base fornisce una struttura iniziale. La memoria locale fornisce adattamento al contesto reale. Nessuna delle due è sufficiente da sola. Il dataset base aiuta a partire con condizioni validate; la memoria locale aiuta a imparare dalle scelte dell'officina. Separarle permette di avere stabilità e adattamento senza confonderli.

Questa separazione sarà fondamentale anche in prospettiva commerciale. Ogni installazione potrà avere memoria locale e feedback propri, mentre il prodotto potrà ricevere aggiornamenti generali controllati senza mescolare dati sensibili.

7 CAM Knowledge Pack e ciclo di validazione

Il CAM Knowledge Pack è una componente di supporto decisionale. Nel workspace analizzato contiene 13184 record candidati. La parola “candidato” è importante: questi record non diventano automaticamente strategie attive. Vengono proposti, valutati, confermati e validati attraverso un ciclo di vita controllato.

7.1 Da candidato ad attivo

Il lifecycle previsto è:

```
candidate -> proposed -> accepted_in_project -> operator_confirmed ->
shop_validated -> active
```

Sono previsti anche stati alternativi come rejected e deprecated. Questa struttura impedisce che conoscenza non verificata entri direttamente nella produzione. In altre parole, il sistema può conoscere molte cose, ma non deve usarle automaticamente come verità operative.

La regola più importante è che `active=true` non viene importato automaticamente. L’attivazione deriva da uno stato validato e richiede conferma. In una prospettiva industriale questa è una garanzia: il learning deve crescere, ma la validazione deve rimanere controllata.

7.2 Decision support, non automazione cieca

Il CAM Knowledge Pack viene usato per proporre candidati coerenti con il contesto. Ogni proposta porta con sé score, ragioni, rischi, controlli e richiesta di eventuale override umano. Questo trasforma il knowledge pack in uno strumento di supporto, non in un motore che impone decisioni.

Questa impostazione è coerente con la filosofia generale del progetto. DirectGCode non vuole dire all’operatore “questa è la strategia, eseguila”. Vuole dire “questa strategia risulta coerente per queste ragioni; verifica questi rischi; conferma o correggi”.

7.3 Feature flag e prudenza operativa

Nel progetto sono presenti feature flag che permettono di attivare o disattivare parti del supporto knowledge e del learning. Questo è importante per rendere reversibile l’integrazione. In un ambiente produttivo non si introduce una funzionalità AI critica senza poterla spegnere, isolare o riportare a uno stato precedente.

La reversibilità è una qualità ingegneristica. Non tutto deve essere attivo subito. Alcune funzioni possono essere consultive, altre in shadow mode, altre ancora abilitate solo dopo validazione.

7.4 Perché il lifecycle è necessario

Un archivio di record tecnici non è automaticamente conoscenza affidabile. Un record può essere corretto in un contesto e pericoloso in un altro. Il lifecycle serve a trasformare progressivamente un’informazione candidata in una strategia utilizzabile.

Lo stato `candidate` indica che il record esiste ma non deve essere trattato come verità. Lo stato `proposed` indica che il sistema lo ritiene potenzialmente utile. Lo stato `accepted_in_project` segnala che è stato inserito in un progetto. Lo stato `operator_confirmed` indica una conferma umana. Lo stato `shop_validated` richiede una validazione pratica. Solo dopo questa catena un record può diventare `active`.

Questo percorso è coerente con il modo in cui l'officina costruisce esperienza: non basta provare una cosa una volta; serve capire se funziona, in quali condizioni e con quali controlli.

7.5 Attivazione al salvataggio

Una logica interessante è l'attivazione controllata al salvataggio. Il salvataggio del progetto può diventare il momento in cui una strategia già validata viene promossa, ma solo se sono soddisfatti flag e condizioni. In questo modo l'apprendimento è legato a un evento operativo concreto, non a un import automatico.

La scelta è prudente. Il sistema non deve diventare più aggressivo solo perché riceve molti record. Deve diventare più affidabile quando i record attraversano conferma e validazione.

7.6 Reversibilità

Ogni strategia deve poter essere rifiutata, deprecata o disattivata. Questo è importante perché la conoscenza tecnica non è immutabile. Cambiano utensili, macchine, materiali, fornitori, condizioni di staffaggio e obiettivi produttivi. Una strategia valida oggi può non esserlo domani.

La reversibilità rende il sistema più robusto. Un software industriale deve poter tornare indietro senza perdere tutto il lavoro fatto.

8 Learning continuo e retention del know-how

Il learning di DirectGCode non è pensato come addestramento incontrollato. È un processo di retention del know-how. Il sistema osserva cosa è stato proposto, cosa è stato corretto e cosa è stato confermato. Questa informazione viene salvata in modo separato rispetto al dataset base.

8.1 Memorie separate

La separazione delle memorie è uno degli elementi più importanti dell'architettura. Il sistema distingue:

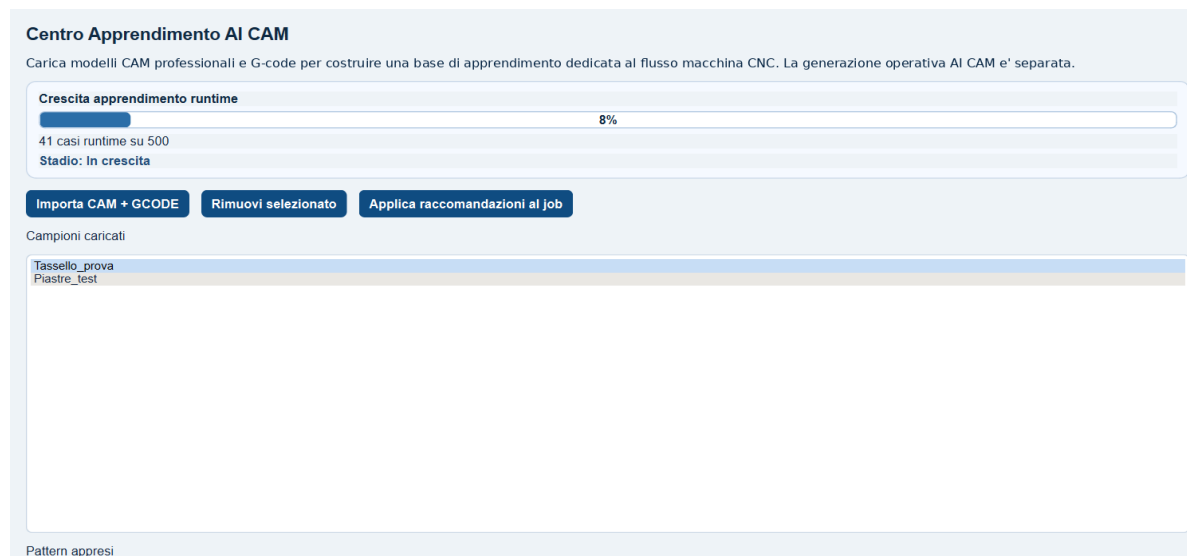
- dataset base read-only;
- casi runtime generati;
- feedback operatore;
- campioni CAM professionali importati;
- knowledge pack candidato.

Questa separazione evita che un errore locale contamini la conoscenza principale. Inoltre permette di distinguere ciò che è generale da ciò che appartiene a un cliente, a un'officina o a un contesto specifico.

8.2 Salvataggio del progetto come momento di apprendimento

Il salvataggio del progetto diventa un momento importante. Il sistema può raccogliere i parametri suggeriti dalla AI, i parametri finali scelti dall'operatore, le correzioni, il livello di confidenza, la configurazione macchina e lo stato di accettazione. In questo modo il progetto salvato non è solo un file operativo, ma anche una traccia di apprendimento.

La logica è semplice: se un operatore corregge sistematicamente una certa proposta, quella correzione ha valore. Se una strategia viene confermata e poi validata, anche quella conferma ha valore. Il sistema deve imparare da questi segnali, ma senza perdere il controllo della qualità.



Pannello di apprendimento AI CAM con crescita runtime, campioni caricati e applicazione raccomandazioni al job.

8.3 Know-how non strutturato e know-how persistente

Uno dei problemi più concreti nelle officine è la dispersione del know-how. Molti file CAM contengono esperienza, ma spesso non sono facili da interrogare. Molti programmi macchina funzionano bene, ma nessuno ha più il contesto completo del perché siano stati fatti in quel modo.

DirectGCode prova a spostare una parte di questa conoscenza in una forma persistente. Non significa automatizzare tutto. Significa costruire memoria tecnica: casi, pattern, feedback, warning, scelte confermate. Una memoria che può essere richiamata quando un nuovo lavoro presenta condizioni simili.

8.4 Feedback come dato tecnico

Il feedback operatore non deve essere trattato come una nota generica. È un dato tecnico. Se l'operatore cambia un avanzamento, sostituisce un utensile, riduce una profondità o rifiuta una strategia, sta fornendo una informazione preziosa. Il sistema deve registrare cosa è stato proposto e cosa è stato scelto.

La differenza tra suggerimento AI e scelta finale è particolarmente importante. Non basta sapere il valore finale; bisogna sapere anche se è stato confermato, corretto o forzato. Questo consente di capire se la AI sta migliorando o se ripete errori.

8.5 Tipi di apprendimento

Il learning può essere letto su tre livelli:

Livello	Descrizione	Uso nel sistema
Apprendimento da campioni	Import di esempi CAM professionali e programmi associati	Costruzione di pattern e casi di riferimento
Apprendimento runtime	Casi generati durante l'uso reale	Adattamento progressivo al contesto operativo
Feedback operatore	Conferme, correzioni e rifiuti	Miglioramento della confidenza e del ranking

Questi livelli hanno pesi diversi. Un campione professionale può essere molto utile, ma non basta se non viene contestualizzato. Un feedback operatore può essere locale, ma molto rilevante per quella officina. Il sistema deve saper distinguere fonte e valore dell'informazione.

8.6 Memoria che non sostituisce l'esperienza

La memoria tecnica non sostituisce l'esperienza dell'operatore. La rende più riutilizzabile. Un operatore esperto continuerà a vedere cose che il sistema non vede: rumore macchina, rigidità reale, utensile consumato, materiale non conforme, urgenza del lavoro. Però il sistema può ridurre il lavoro ripetitivo e ricordare ciò che è già stato provato.

Questo è uno dei punti più forti dell'AI industriale pragmatica: non promette di eliminare il mestiere, ma di non disperderlo.

9 Generazione G-code e integrazione officina

La generazione del G-code è la parte in cui la proposta diventa output operativo. Per questo il progetto mantiene un approccio locale, validato e controllato.

9.1 Postprocessor locale

Il postprocessor produce codice ISO/G-code a partire dal toolpath. Questa fase deve essere prevedibile. L'AI può suggerire la strategia, ma la traduzione in codice macchina non può essere lasciata a un testo generato liberamente. Deve passare da regole e strutture controllabili.

Il sistema prevede validazione del progetto, stima dei tempi ciclo e controlli sul codice generato. La validazione può includere coerenza dei parametri, presenza di istruzioni fondamentali, formato del programma, compatibilità con l'output richiesto e controlli di sicurezza di base.

9.2 Setup sheet e pacchetto officina

Un output CAM utile non è solo il G-code. L'officina ha bisogno di informazioni: utensili, origine, grezzo, parametri, note, controlli, eventuali warning. Il setup sheet serve a trasferire queste informazioni in modo ordinato.

La produzione reale vive di comunicazione. Un programma corretto ma non accompagnato da informazioni chiare può generare errori di attrezzaggio, fraintendimenti o tempi morti. Per questo DirectGCode considera il setup sheet come parte del flusso operativo.

9.3 Trasferimento verso centro di lavoro CNC

Il trasferimento verso la macchina CNC viene trattato come fase separata. Nel progetto sono previsti controlli su formato, caratteri, terminazioni riga, struttura programma e modalità di invio. La logica generale è che il programma deve essere verificato prima di uscire dal sistema.

La scelta di descrivere questa parte in modo generico è voluta. Il sistema deve poter lavorare con profili macchina diversi. Ciò che conta è il principio: validazione locale, trasferimento controllato, logging e nessuna esposizione non necessaria dei canali macchina in rete.

9.4 Perché non generare direttamente G-code con un modello libero

La tentazione di usare un modello generativo per scrivere direttamente G-code è forte, ma in un contesto industriale è rischiosa. Il G-code deve rispettare formati, sequenze, limiti macchina, coordinate, compensazioni, origini e funzioni ausiliarie. Un errore apparentemente piccolo può avere conseguenze fisiche.

DirectGCode preferisce un approccio più controllato: la AI propone a monte, il motore CAM genera a valle. Questa separazione è una forma di sicurezza. Il modello non "scrive liberamente" il programma; contribuisce alla decisione tecnica che poi viene tradotta da componenti deterministici.

9.5 Validazione prima dell'esportazione

Prima dell'esportazione devono essere controllati almeno tre aspetti: coerenza del progetto, coerenza del percorso e coerenza del codice. Il progetto deve avere parametri sufficienti, il percorso deve essere interpretabile e il codice deve rispettare una struttura minima.

In una fase industriale matura, questa validazione può essere estesa con controlli più avanzati: limiti asse, collisioni, envelope macchina, limiti utensile, compatibilità postprocessor, confronto con setup sheet e check-list qualità.

9.6 Documentazione operativa

Il setup sheet è un ponte tra software e officina. Deve contenere solo informazioni utili, in modo leggibile. Se diventa troppo lungo o confuso, perde valore. Se è troppo povero, non supporta l'operatore. L'equilibrio corretto è fornire dati essenziali: utensile, operazione, origine, parametri, note, warning e controlli richiesti.

La documentazione operativa è anche una forma di standardizzazione. Permette di ridurre la dipendenza da comunicazioni verbali e rende più chiaro il trasferimento tra programmazione e macchina.

10 Sicurezza, governance AI e separazione dati

In un progetto AI industriale la sicurezza non può essere aggiunta alla fine. Deve influenzare l'architettura fin dall'inizio. DirectGCode affronta la sicurezza su tre livelli: dati, output e infrastruttura.

10.1 Protezione dei dati cliente

I dati cliente, le geometrie, i programmi e i feedback possono contenere informazioni riservate. Per questo il progetto prevede una separazione tra learning locale, dataset globale e conoscenza proprietaria. L'eventuale condivisione verso un learning globale deve essere opt-in, non automatica.

Questo punto è cruciale anche in prospettiva commerciale. Un cliente non può ricevere una copia del sistema contenente dati o strategie proprietarie di altri. Allo stesso tempo il produttore del software non deve distribuire il proprio modello completo e il proprio dataset globale senza controllo.

10.2 Output critici human-in-the-loop

Il G-code è un output critico. Può muovere assi, utensili e mandrino. Per questo non deve essere alterato da un componente cloud o AI senza approvazione operatore. L'approccio human-in-the-loop non è solo una scelta etica, ma una misura tecnica di sicurezza.

L'operatore deve poter vedere cosa viene proposto, modificare i valori, verificare la simulazione, leggere warning e confermare. Il sistema non elimina la responsabilità tecnica: la supporta.

10.3 Superficie di attacco ridotta

Mantenere la produzione locale riduce la superficie di attacco. I componenti API e Docker possono essere utili per industrializzare alcune funzioni, ma non devono esporre direttamente canali macchina o dati sensibili. Il trasferimento verso il CNC rimane locale e separato.

Questa scelta è coerente con una visione prudente dell'AI industriale: aumentare capacità e tracciabilità senza aprire inutilmente punti di rischio.

10.4 Proprietà del dato e fiducia commerciale

La sicurezza dei dati non è solo un tema tecnico: è un tema commerciale. Un cliente deve sapere che i propri file, le proprie geometrie e i propri programmi non vengono usati per addestrare altri ambienti senza consenso. Allo stesso tempo, chi sviluppa DirectGCode deve proteggere il proprio patrimonio tecnico.

La separazione tra dataset globale, memoria locale e feedback opt-in permette di costruire un modello di fiducia. Ogni parte sa cosa rimane locale, cosa può essere condiviso e cosa viene aggiornato centralmente.

10.5 AI governance concreta

La governance AI, in questo progetto, non è un documento astratto. È incorporata in scelte operative:

- output critici con controllo umano;
- feature flag per abilitazione progressiva;
- learning separato dal dataset base;
- stato active non importato automaticamente;
- logging degli eventi di apprendimento;
- shadow mode per modelli non ancora validati;
- canali macchina non esposti inutilmente in rete;
- aggiornamenti gestiti in modo reversibile.

Queste scelte rendono il sistema più difendibile tecnicamente. Non eliminano la necessità di procedure aziendali, ma creano una base architettuale coerente.

10.6 Continuità produttiva

La produzione non può dipendere da un componente non disponibile. Per questo il sistema deve avere fallback, modalità conservative e possibilità di lavorare anche senza alcune funzioni AI avanzate. Se un modello ML non è disponibile, il ranking può usare euristiche. Se il servizio API non è attivo, la GUI locale deve poter continuare a funzionare. Se il knowledge pack è disattivato, il dataset base e le regole devono rimanere utilizzabili.

Questa capacità di degradare in modo controllato è una caratteristica fondamentale dei sistemi industriali.

11 Test, risultati e limiti

Ogni progetto tecnico deve distinguere tra prototipo, sistema in sviluppo e prodotto industriale validato. DirectGCode si colloca in una fase avanzata di sviluppo tecnico, con componenti funzionanti e test mirati, ma con limiti ancora da trattare prima di un'adozione produttiva estesa.

11.1 Dati rilevati all'export

Alla data dell'export tecnico utilizzato per questo documento risultano:

Indicatore	Valore
Condizioni tecniche validate nel dataset AI CAM base	2000
Record candidati nel CAM Knowledge Pack	13184
Campioni AI CAM importati	2
Casi runtime generati	39
Feedback operatore salvati	2
Test mirati eseguiti sui servizi AI CAM principali	67 passati

Questi numeri indicano che il progetto non è solo concettuale. Esiste un ambiente operativo, esistono dataset, esistono casi runtime, esiste una knowledge base e ci sono test automatizzati. Allo stesso tempo non devono essere interpretati come certificazione completa del sistema su ogni condizione di lavorazione possibile.

11.2 Valore dei test

I test passati sui servizi AI CAM principali servono a verificare componenti come caricamento dataset, pianificazione STEP AI CAM e ranking utensili. Sono test importanti perché proteggono da regressioni e permettono di introdurre nuove funzioni senza rompere ciò che funziona.

In un sistema che deve crescere nel tempo, i test non sono un dettaglio. Sono la base per rendere possibile l'evoluzione. Ogni nuova integrazione AI, ogni feature flag e ogni componente Docker/API dovrebbe essere introdotto mantenendo la possibilità di tornare indietro.

11.3 Limiti attuali

I limiti principali non sono da nascondere. Al contrario, riconoscerli aumenta la credibilità del progetto. Tra i limiti più importanti:

- la validazione industriale deve essere estesa su più materiali, macchine e geometrie;
- il learning runtime richiede più feedback operatore per diventare statisticamente solido;
- alcune funzioni headless sono volutamente conservative;
- la generazione da STEP deve essere testata su geometrie sempre più complesse;
- il ranking ML deve rimanere in shadow mode finché non ha evidenze sufficienti;
- la knowledge candidate non deve essere attivata senza percorso di validazione.

Questi limiti non indeboliscono il progetto. Definiscono il percorso di maturazione.

11.4 Test unitari, test funzionali e test industriali

I test automatizzati sono necessari, ma non bastano. In un progetto come DirectGCode servono diversi livelli di verifica:

Tipo di test	Obiettivo	Esempio
Unitario	Verificare un servizio isolato	Caricamento dataset, ranking utensile, parsing configurazione
Funzionale	Verificare un flusso software	Import modello, proposta AI CAM, generazione output
Regressione	Evitare che una modifica rompa funzioni già valide	Suite test dopo integrazione nuove feature
Simulazione	Verificare il percorso prima dell'officina	Preview toolpath, stima tempi, warning
Industriale	Validare su macchina e materiale reali	Prove controllate con operatore esperto

I 67 test passati sono quindi un punto di partenza tecnico. La validazione industriale richiede un piano più ampio, con casi reali, materiali differenti, geometrie rappresentative e raccolta sistematica dei feedback.

11.5 Metriche future

Per maturare il sistema sarà utile misurare indicatori come:

- percentuale di proposte AI accettate senza modifica;
- percentuale di proposte corrette dall'operatore;
- tipo di correzioni più frequenti;
- tempo medio di preparazione job;
- numero di warning utili confermati;
- accuratezza della stima tempo ciclo;
- riduzione degli errori di setup;
- coerenza tra casi richiamati e scelte finali;
- frequenza di attivazione di strategie dopo validazione.

Queste metriche permetteranno di trasformare il progetto da sistema promettente a prodotto misurabile.

11.6 Validazione prudente

La validazione deve partire da casi controllati. Prima geometrie semplici, poi feature più complesse. Prima materiali conosciuti, poi materiali più critici. Prima strategie conservative, poi ottimizzazioni. Questa progressione riduce il rischio e permette di costruire fiducia.

Un errore comune nei progetti AI è voler dimostrare subito il massimo. In officina è più utile dimostrare affidabilità progressiva.

12 Industrializzazione e sviluppi futuri

La fase successiva di DirectGCode riguarda l'industrializzazione. Questo non significa semplicemente aggiungere funzioni, ma costruire un sistema distribuibile, aggiornabile, sicuro e mantenibile.

12.1 Docker e API

Docker può essere utile per isolare servizi separabili: API, validazione headless, stima tempi, eventuali servizi di knowledge e componenti non direttamente legati alla GUI. La GUI desktop non deve necessariamente essere containerizzata. Anzi, in un contesto officina può essere più sensato mantenerla locale e usare container solo per servizi di supporto.

L'obiettivo è evitare una migrazione aggressiva che rompa il progetto esistente. L'industrializzazione deve essere progressiva: prima analisi dell'architettura, poi identificazione dei componenti separabili, poi containerizzazione controllata, poi test, poi eventuale predisposizione per orchestrazione futura.

12.2 Predisposizione a Kubernetes

Kubernetes può diventare utile quando il sistema dovrà scalare più servizi, ambienti cliente, versioni, API, knowledge sharing controllato o infrastrutture cloud/on-premise. Non è necessario per far funzionare il desktop in officina. È una prospettiva di crescita, non un requisito iniziale.

La logica corretta è progettare oggi in modo che una futura migrazione sia possibile, senza introdurre complessità non necessaria. In pratica: confini chiari tra servizi, configurazioni esterne, logging, health check, versioning, test e separazione dati.

12.3 Aggiornamenti e pacchetti

In una prospettiva commerciale il sistema dovrà essere modulare. Alcune funzioni potranno appartenere al prodotto base, altre a pacchetti avanzati. Gli aggiornamenti dovranno essere caricabili in modo controllato, verificabile e reversibile.

Questo aspetto è collegato al learning. Il cliente non deve ricevere il dataset globale completo, né contenere dati di altri clienti. Allo stesso tempo il software deve poter beneficiare di conoscenza comune, aggiornata e validata. La soluzione dovrà prevedere canali separati per aggiornamenti software, knowledge pack, modelli e feedback opt-in.

12.4 Visione a lungo termine

La visione finale non è solo un CAM assistito. È un ecosistema AI industriale capace di:

- conservare il know-how produttivo;
- guidare scelte CAM complesse;
- supportare operatori meno esperti;
- ridurre tempi di programmazione;
- aumentare tracciabilità e qualità;

- connettersi a sistemi officina e supervisione;
- mantenere controllo umano sugli output critici.

DirectGCode è un passo in questa direzione: non una promessa generica di automazione, ma un modello concreto di AI applicata alla produzione meccanica.

12.5 Versionamento e rollback

Un prodotto industriale deve poter essere aggiornato senza paura. Ogni modifica importante deve essere versionata, testata e reversibile. Questo vale per il codice, ma anche per knowledge pack, modelli, configurazioni e feature flag.

L'uso disciplinato di repository, branch, tag, changelog e suite di test permette di mantenere il controllo. In caso di aggiornamento problematico, deve essere possibile tornare a una versione precedente. Questo è particolarmente importante quando il software entra in ambienti produttivi dove un fermo ha costo reale.

12.6 Pacchetti funzionali

La suddivisione futura in pacchetti può seguire una logica modulare:

Pacchetto	Contenuto possibile	Criterio di rilascio
Base	Progetto, import, toolpath base, postprocessor, setup sheet	Funzioni essenziali e stabili
AI CAM	Suggerimenti, dataset, retrieval, warning	Controllo operatore obbligatorio
Learning	Campioni, runtime memory, feedback	Separazione dati e opt-in
Knowledge Pack	Strategie candidate e lifecycle	Validazione progressiva
API/Industrial	Servizi headless, health check, integrazione sistemi	Ambito controllato e testato

Questa struttura permette di vendere o distribuire funzioni avanzate senza confondere il prodotto base con componenti ancora sperimentali.

12.7 Aggiornamenti knowledge e modelli

Gli aggiornamenti del knowledge devono essere distinti dagli aggiornamenti software. Un bug fix del programma non è la stessa cosa di un nuovo set di strategie candidate. Allo stesso modo, un modello ML aggiornato deve essere accompagnato da versione, note, test e possibilità di rollback.

Questo è un punto chiave per evitare che il sistema diventi opaco. Ogni componente che influenza una decisione CAM deve essere identificabile.

13 Conclusioni

DirectGCode dimostra che l'intelligenza artificiale può essere integrata nei processi CAM in modo pragmatico, senza cadere nell'idea rischiosa dell'automazione cieca. Il valore del progetto non sta nel dire che la AI può generare un programma CNC da sola, ma nel mostrare come una architettura ibrida possa supportare decisioni tecniche complesse mantenendo controllo, tracciabilità e validazione umana.

Il progetto unisce elementi diversi: feature engineering geometrico, dataset tecnico, retrieval di casi, CAM Knowledge Pack, ranking utensili, calcolo parametri, postprocessor locale, feedback operatore e governance del learning. Questa combinazione crea un sistema che può crescere con l'esperienza, ma senza rinunciare alla prudenza necessaria in officina.

La tesi tecnica di fondo è questa: l'AI industriale utile non deve sostituire il mestiere, ma renderlo più trasferibile, più strutturato e più controllabile. In un settore dove il know-how è spesso disperso tra persone, file e abitudini operative, DirectGCode prova a trasformare una parte di quell'esperienza in memoria tecnica persistente.

Il percorso non è concluso. Servono ulteriori validazioni, più casi reali, più feedback, più test su geometrie e materiali diversi, e una industrializzazione progressiva. Ma la direzione è chiara: un sistema AI CAM locale, spiegabile, human-in-the-loop e generalizzabile a diversi centri di lavoro CNC.

Il risultato finale non è solo un software. È un modo diverso di pensare la programmazione CNC: meno dipendente dalla memoria individuale, più orientato alla conoscenza condivisa, più sicuro nell'adozione dell'AI e più vicino alla realtà dell'officina.

Appendice A - Glossario operativo

AI CAM

Sistema di supporto alla programmazione CAM che utilizza intelligenza artificiale, regole tecniche e dati storici per proporre strategie di lavorazione.

CAM Knowledge Pack

Archivio strutturato di record candidati relativi a strategie, utensili, rischi e controlli. Non viene attivato automaticamente senza validazione.

Case library

Libreria di casi tecnici utilizzata dal sistema per confrontare il lavoro corrente con esperienze precedenti o condizioni validate.

Centro di lavoro CNC

Macchina utensile a controllo numerico utilizzata per operazioni di fresatura, foratura, contornatura e lavorazioni simili.

Feature engineering geometrico

Processo di trasformazione di una geometria CAD in elementi tecnici interpretabili: piani, tasche, facce, regioni aperte, profondità e accessibilità.

G-code

Linguaggio di programmazione utilizzato per descrivere movimenti e funzioni macchina in ambito CNC.

Human-in-the-loop

Approccio in cui l'operatore umano rimane parte del processo decisionale, specialmente per output critici.

Postprocessor

Componente che trasforma il toolpath in codice macchina compatibile con un determinato controllo CNC o profilo macchina.

RAG tecnico

Approccio di retrieval applicato al CAM: il sistema recupera casi e conoscenze tecniche pertinenti per supportare la generazione della proposta.

Shadow mode

Modalità in cui un modello o servizio AI produce raccomandazioni senza forzarne l'applicazione. Serve a osservare e validare il comportamento.

Toolpath

Percorso utensile calcolato per eseguire una lavorazione.

Appendice B - Criteri editoriali della versione pubblicabile

Questa versione del documento è stata impostata con criteri adatti alla stampa, alla rilegatura e alla pubblicazione professionale:

- uso di terminologia macchina generica, senza vincolo a marchi specifici;
- esclusione di codice sorgente e dettagli proprietari sensibili;
- descrizione dell'architettura per livelli funzionali;
- valorizzazione del contesto industriale reale;
- evidenza del controllo umano sugli output critici;
- separazione tra progetto tecnico indipendente e titolo accademico;
- figure dimostrative normalizzate per evitare riferimenti non generalizzabili.

Appendice C - Dati tecnici dell'export utilizzato

Il documento è basato sul report tecnico esportato dal progetto DirectGCode in data 2026-05-26. Le schermate incluse sono state generate dal codice UI reale del progetto, usando un progetto salvato e dati locali rappresentativi. Alcune sezioni dei pannelli sono state rese visibili con dati dimostrativi o rappresentativi per facilitare la lettura tecnica.

Il contenuto è stato rielaborato in forma divulgativa tecnica, con l'obiettivo di rendere comprensibile l'architettura senza esporre informazioni riservate o strategie proprietarie.